Confluent Platform SQL Deployment via Flink SQL Client — User Event Enricher PTF UDF

> This example deploys the **User Event Enricher** PTF UDF (source in examples/ptf_udf/java/) by submitting SQL statements through the **Flink SQL Client** running directly on the JobManager pod.

Table of Contents

- 1.0 Overview
  - 1.1 How this differs from the other deployment paths
- 2.0 How it works
  - 2.1 JAR delivery
  - 2.2 Kafka connector
  - 2.3 Statement flow
- 3.0 Prerequisites
- 4.0 How to run
  - 4.1 Deploy
  - 4.2 Monitor
  - 4.3 Tear down
- 5.0 Resources

1.0 Overview

1.1 How this differs from the other deployment paths

Aspect	Confluent Platform SQL Client (this example)	Confluent Cloud Terraform
Where it runs	Confluent Platform + Minikube	Confluent Cloud
How SQL is submitted	`sql-client.sh -f` on the JobManager pod	`confluent_flink_statement` Terraform resources
UDF JAR delivery	`kubectl exec` to Flink pods	`confluent_flink_artifact` (uploaded to CC)
Entry point	`make deploy-cp-ptf-udf`	`make deploy-cc-ptf-udf`
Requires code compilation	✅ (Java + Gradle for UDF JAR)	✅ (Java + Gradle for UDF JAR)
Statement lifecycle	Managed by Flink session cluster	Managed by Terraform state
Same codebase for both?	✅ (same Java UDF code, different Terraform vs SQL Client for deployment)	✅ (same Java UDF code, different Terraform vs SQL Client for deployment)

2.0 How it works

2.1 JAR delivery

On Confluent Platform there is no artifact store — the JAR must be physically present on the Flink pods.

The deploy script copies the fat JAR (built from examples/ptf_udf/java/) to `/opt/flink/usrlib/user-event-enricher.jar` on every JobManager and TaskManager pod using `kubectl exec`. The `CREATE FUNCTION ... USING JAR` statement then references this pod-local path.

2.2 Kafka connector

The CP Flink base image does not include the Kafka SQL connector. The FlinkDeployment uses two **init containers** to assemble `/opt/flink/lib/` at pod startup:

1. **copy-stock-lib** — copies the stock Flink lib JARs from the image into a shared `emptyDir` volume
2. **fetch-kafka-connector** — downloads `flink-sql-connector-kafka` from Maven Central into the same volume

The main Flink container then mounts this volume at `/opt/flink/lib/`, so the Kafka connector is on the classpath when the JVM starts.

2.3 Statement flow

The script pre-creates Kafka topics, then executes all SQL in a single `sql-client.sh -f` session on the JobManager pod:

```
┌──────────────────────────────────────────────────────────────────┐
│  Pre-step: kafka-topics --create user_events, enriched_events    │
│                                                                   │
│  Step 1:  DROP TABLE IF EXISTS user_events              → OK      │
│  Step 2:  CREATE TABLE user_events (... WITH kafka ...) → OK      │
│  Step 3:  INSERT INTO user_events VALUES (sample data)  → submitted │
│  Step 4:  DROP TABLE IF EXISTS enriched_events          → OK      │
│  Step 5:  CREATE TABLE enriched_events (... WITH kafka) → OK      │
│  Step 6:  CREATE FUNCTION user_event_enricher           → OK      │
│           USING JAR '/opt/flink/usrlib/...'                       │
│  Step 7:  INSERT INTO enriched_events                   → submitted │
│           SELECT ... FROM TABLE(user_event_enricher())            │
└──────────────────────────────────────────────────────────────────┘
```

Step 7 is a **long-running streaming job**. It runs continuously, reading from `user_events` and writing enriched output to `enriched_events`.

3.0 Prerequisites

- macOS with Homebrew and Docker Desktop running
- Java 17+
- Confluent Platform and Flink stack already deployed:

```
make install-prereqs       # installs tooling (docker, kubectl, minikube,
helm, gradle)
make cp-up                 # Minikube → CFK Operator → Kafka + SR +
Connect + C3 + Kafka UI
make cp-watch              # watch pods come up (Ctrl+C when all Running)
make flink-up              # cert-manager → Flink Operator → CMF → Flink
session cluster
make flink-status          # verify Flink pods are Running
```

4.0 How to run

All commands are run from the **project root** (where the `Makefile` lives).

4.1 Deploy

A single target builds the UDF JAR, copies it to the Flink pods, and executes all SQL:

```
make deploy-cp-ptf-udf
```

Behind the scenes this runs:

Step	What it does
1	`./gradlew clean shadowJar` — builds the UDF fat JAR from `examples/ptf_udf/java/`
2	`kubectl exec` — copies the JAR to all JobManager and TaskManager pods
3	`kafka-topics --create` — pre-creates Kafka topics (`user_events`, `enriched_events`)
4	`sql-client.sh -f` — executes all SQL statements in a single session on the JobManager pod

4.2 Monitor

Open the Flink Dashboard to see the running enrichment job:

```
make flink-ui              # opens http://localhost:8081
```

4.3 Tear down

To stop the running enrichment job and drop all tables and functions:

```
make teardown-cp-ptf-udf
```

This cancels any running Flink jobs via the Flink REST API, then submits `DROP FUNCTION` and `DROP TABLE` statements.

5.0 Resources

- Flink SQL Client
- Flink Kafka Connector
- Create a User-Defined Function (Confluent Cloud)